



East West University

Title

Project Report

Course Code: CSE 207

Course Title: Data Structure and Algorithm

Semester: Summer 2025

Section: 10

Course Instructor

Dr. Md. Manowarul Islam

Adjunct faculty

Department of Computer Science & Engineering

Submitted by

Name	ID
Anjuman Ara	2024-1-60-279
Rakib Al Hasan	2024-1-60-143

Combined Infix & Postfix Expression Evaluator

The purpose of this project is to analyze mathematical equations written in both infix and comma-separated postfix notation using a C program. Along with error management for faulty or unsupported expressions, the application also enables conversion between Infix, Postfix, and Prefix expressions. It illustrates parsing and evaluating expressions using stack and queue-based techniques.

Objectives: to create a calculator that can handle both postfix (comma-separated) and infix notations. to offer prefix, postfix, and infix expression conversion. to report restrictions and handle expressions with or without variables. to put in place reliable error handling (mismatched parenthesis, division by zero, and invalid expressions). to enable several evaluations in order to make the software interactive.

Key Features: **Infix Expression Handling:** validation, evaluation in the absence of variables, and conversion to Postfix and Prefix. Evaluation of expressions written in comma-separated format is supported by postfix (comma-separated) handling. **Evaluation Engine:** Postfix and Prefix are evaluated using stack-based and queue-based methods, respectively. **Error Handling:** Deals with incomplete expressions, division by zero, incorrect characters, and misaligned parentheses. **User Engagement:** asks users repeatedly if they would like to carry out more procedures.

Methodology:

The project is based on traditional expression assessment algorithms: **Stacks:** Used to transform Infix to Postfix and evaluate Postfix expressions. Prefix expressions are evaluated using queues. **Tokenization:** The application divides the input into tokens and analyzes each one in turn for Postfix with commas. **Validation:** Prior to processing, the application looks for unlawful operators, misaligned parenthesis, and invalid characters. **User Loop:** The user is asked to either continue or stop after every action.

Tools and Technologies: C is the programming language. **Data Structures:** Queues and Stacks (manually created using linked lists) **Environment of Development:** Code:: GCC compiler/Blocks , , , and are libraries.

Flowchart:

The following is the order in which the program flows: An expression is entered by the user. The application determines if it is Comma-separated Postfix or Infix. It evaluates directly if Postfix is used, and converts to Postfix and Prefix if Infix is used. Along with any error warnings pertaining to invalid inputs, the results are shown.

The user is given an exit or continue prompt. One possible addition to the presentation slides is a comprehensive flowchart diagram.

Conclusion:

Using C, this project effectively creates a hybrid infix and postfix expression evaluator. It offers an interactive user experience, gracefully manages failures, and effectively illustrates stack and queue data structures. The project has uses in academic data structure and algorithm learning, compiler design, and mathematical expression processing.